Emergency Alert App - Complete Product Specification for Cursor

Project Overview

A cross-platform emergency alert application for Android and iOS. The app allows users to quickly trigger an SOS alert, securely transmit their location to a backend, and notify trusted contacts or nearby users. The design prioritizes user safety, privacy, encryption, reliability, and compliance with platform rules.

Core User Flow

1. User registers and verifies phone/email. 2. User grants location and notification permissions. 3. User configures emergency contacts. 4. User triggers an emergency alert using an approved trigger. 5. App captures location and alert type. 6. Alert is encrypted and sent to the backend. 7. Backend authenticates the request. 8. Nearby users and/or trusted contacts receive notifications. 9. User may cancel a false alarm within a short grace period.

Mobile Technology Stack

Flutter application for Android and iOS. Recommended packages: - firebase_auth - firebase_messaging - geolocator - flutter_secure_storage - local_auth - flutter_background_service

Backend Technology Stack

Node.js + Express API. PostgreSQL or MongoDB database. Redis Geo indexing for nearby-user discovery. Firebase Cloud Messaging for push notifications. HTTPS/TLS 1.3 for secure communications.

Authentication

Phone verification via OTP. Email verification. JWT-based authentication. Device registration and token management.

Permissions

Request: - Location While Using App - Background Location (where supported) - Notifications Explain clearly why each permission is required.

Emergency Trigger System

iOS: - In-app panic button. - Siri Shortcuts. - Other approved platform mechanisms. Android: - In-app panic button. - Approved background services. - Hardware-trigger integrations where supported by the OS.

Alert Payload

Fields: - User ID - Latitude - Longitude - Alert Type - Timestamp - Device ID

Encryption

Use hybrid encryption: - AES-256-GCM for payload encryption. - RSA-4096 public key encryption for AES key exchange. - Signed requests for integrity verification.

Backend API Endpoint

POST /api/v1/emergency Flow: - Verify JWT - Validate signature - Decrypt payload - Store alert - Locate nearby users - Send push notifications

Database Design

Users Table: - id - name - phone - email - publicKey - deviceToken Alerts Table: - id - userId - latitude - longitude - alertType - status - timestamp

User Interface

Minimal UI: - Large SOS button - Status indicators - Emergency contacts - Alert history - False-alarm cancellation screen

Nearby User Notifications

Backend determines users within a configurable radius (for example 2–5 km). Push notifications contain: - Alert type - Approximate distance - Timestamp - Link to map view

Security Requirements

- TLS 1.3 - Secure device key storage - Rate limiting - Replay attack protection - Fraud detection - Encrypted sensitive records

Testing Requirements

Functional Testing: - Registration - Login - Alert creation - Alert cancellation - Notification delivery Security Testing: - JWT validation - Encryption validation - Penetration testing Performance Testing: - High user volumes - Simultaneous alert scenarios

Future Features

- Live location sharing - Audio recording attachment - Photo/video evidence - Wearable support - Administrative dashboard - Emergency responder integration